GoldenSetAuditor

GoldenSetAuditor / Interview Defense Document

Sidharth Kriplani

Document v1.0 · Sourced exclusively from repo files

Label	Meaning
[IMPLEMENTED]	Code exists, runs, produces verified artifact or DB row
[SIMULATED]	Deliberately synthetic scenario; simulation logic is real code
[PARTIALLY VERIFIED]	Code exists but verification relies on internal consistency only
[FUTURE]	Not built — explicitly absent from the repo

GoldenSetAuditor Interview Defense Document v2.0

Complete Edition — LLM/RAG Evaluation Dataset Quality Auditing

SECTION 1 — PROJECT IDENTITY

1.1 One-Sentence Summary

GoldenSetAuditor audits the evaluation datasets used to benchmark LLM and RAG systems — checking for conflicting labels, exact duplicates, weak reference answers, ambiguous prompts, over-easy examples, near-duplicate pairs, and category coverage gaps — before a single benchmark score is trusted.

1.2 The Problem This Solves

Nobody questions the golden set. It's treated as ground truth — the fixed reference that tells you whether your model improved. But golden sets are assembled by humans under deadline pressure, from domain knowledge that isn't always consistent or independently reviewed. The same question appears twice with different expected answers. A near-trivial question makes up a third of a category. A reference answer is one word. An ambiguous pronoun in a prompt means no single correct answer exists.

None of this is visible in the benchmark score itself. A bad golden set doesn't produce obviously wrong numbers — it produces confidently wrong ones. GoldenSetAuditor breaks that circularity: it audits the evaluation dataset before any benchmark score is published.

1.3 Truth Boundary

GoldenSetAuditor audits dataset quality. It does not evaluate model answers. It does not check whether expected answers are factually correct, whether the evaluation metric (exact match, ROUGE, LLM-judge) is appropriate, whether the golden set covers the production query distribution, or whether pretraining contamination has occurred. These limitations are stated explicitly on every output report and cannot be suppressed.

SECTION 2 — THE SEVEN CHECKS

2.1 Check Architecture

Group	Check	Method	Max Status
Structural	Conflicting labels	Same prompt → different expected answers	**FAIL**
Structural	Exact duplicates	Same prompt → same expected answer	WARN
Content	Weak expected answer	Meaningful token count after stopword filter	WARN
Content	Ambiguous prompt	Anaphoric reference + multi-question heuristic	WARN
Content	Over-easy example	Token Jaccard between prompt and expected answer	WARN
Content	Near-duplicate scan	Pairwise token Jaccard across all prompt pairs ($O(n^2)$)	WARN or INSUFFICIENT_INPUT
Coverage	Category coverage	Example count per category value	WARN or INSUFFICIENT_INPUT

2.2 Conflicting Labels [IMPLEMENTED — Only FAIL check]

Two rows with identical normalised input text but different expected answers represent a structural impossibility in evaluation. The ground truth is undefined: if the model produces the answer from row A, is it correct or incorrect? This ambiguity poisons every metric computed on the evaluation set. Conflicting labels produce FAIL, not WARN — there is no legitimate reason for the same prompt to have two different correct answers in a static evaluation set.

Normalisation before comparison: lowercase, strip whitespace, collapse multiple spaces, remove leading/trailing punctuation. This prevents near-identical prompts (differing only in a trailing period) from escaping detection.

2.3 Exact Duplicates [IMPLEMENTED]

Exact duplicates — same prompt, same expected answer — are redundant examples that inflate example counts without adding evaluation coverage. They produce WARN (not FAIL) because duplicates don't break the evaluation, they just waste evaluation budget. A golden set with 100 examples where 30 are duplicates is effectively a 70-example set with overcounted category representations.

2.4 Weak Expected Answer [IMPLEMENTED]

Expected answers with fewer than `min_expected_words` (default: 3) meaningful tokens (after stopword filtering) are flagged WARN. Single-word or very short answers ("Yes", "No", "True", "False", "2") create evaluation problems: they are either trivially correct (any model that says "Yes" passes) or trivially incorrect (any answer phrased differently fails). Neither is useful for measuring model capability.

The stopword filter removes articles, prepositions, and common filler words before counting meaningful tokens — an expected answer of "the answer is yes" counts as 1 meaningful token, not 4.

2.5 Ambiguous Prompt [IMPLEMENTED]

Prompts are flagged WARN for two ambiguity patterns:

Anaphoric references: Prompts containing pronouns ("it", "they", "this", "that", "these", "those") in short prompts (<50 tokens) where the referent is not established in the prompt text. These prompts depend on context that may not be in the retrieval result.

Multiple question marks: Prompts containing more than one question mark are asking multiple questions. Evaluation against a single expected answer is ambiguous — which question is the model supposed to answer?

2.6 Over-Easy Example [IMPLEMENTED]

Token Jaccard between the prompt and the expected answer measures whether the model could answer by copying from the prompt. Jaccard = |prompt_tokens ∩ answer_tokens| / |prompt_tokens ∪ answer_tokens|. Examples with Jaccard ≥ `oe_threshold` (default: 0.5) are flagged WARN — they test extraction, not reasoning or retrieval quality.

Over-easy examples are not harmful individually, but a golden set dominated by them doesn't distinguish a good RAG system from a simple keyword matcher. They dilute the evaluation's signal-to-noise ratio.

2.7 Near-Duplicate Scan [IMPLEMENTED]

Pairwise token Jaccard is computed across all prompt pairs. Pairs with Jaccard ≥ `nd_threshold` (default: 0.7) are flagged as near-duplicates. Near-duplicate prompts (paraphrased versions of the same question) reduce evaluation diversity: a model that memorizes one phrasing effectively scores well on all paraphrases.

The computational complexity is $O(n^2)$ in the number of examples. For large golden sets (>5,000 examples), the check uses random sampling (1,000 pairs) to keep runtime manageable, returning INSUFFICIENT_INPUT with a note rather than a false PASS.

2.8 Category Coverage [IMPLEMENTED]

For each unique value in the `category_col`, the check verifies that at least `min_category_count` (default: 5) examples are present. Categories with fewer examples produce per-group metrics that have high variance — a category with 2 examples where the model answers both correctly achieves 100% precision, but this tells us nothing about true category-level model performance. Flagged WARN with the category name and count as evidence.

If no `category_col` is provided, the check returns INSUFFICIENT_INPUT — the distinction matters because "no categories provided" is not the same as "all categories have adequate coverage."

SECTION 3 — API AND USAGE

3.1 Core API

```
from goldensetauditor import GoldenSetAuditor, GoldenSetAuditConfig config = GoldenSetAuditConfig(
question_col="question", expected_answer_col="expected_answer", category_col="category",
id_col="id", min_expected_words=3, oe_threshold=0.5, nd_threshold=0.7, min_category_count=5, )
auditor = GoldenSetAuditor(df=eval_df, config=config) report = auditor.audit()
print(report.overall_status) # FAIL / WARN / INSUFFICIENT_INPUT / PASS report.to_json("audit.json")
report.to_markdown("audit.md") report.to_html("audit.html")
```

3.2 JSONL Input

For teams working with JSONL evaluation files (common in LLM benchmarking):

```
import pandas as pd import json with open("eval_set.jsonl") as f: rows = [json.loads(line) for line
in f] df = pd.DataFrame(rows) auditor = GoldenSetAuditor(df=df, config=config) report =
auditor.audit()
```

SECTION 4 — VERSION HISTORY

4.1 v0.5.0 (Current) — Answer Diversity Check [IMPLEMENTED]

Added answer diversity analysis using three complementary metrics: Shannon entropy over answer tokens (measures vocabulary richness), Type-Token Ratio (TTR = unique tokens / total tokens, measures lexical diversity), and coefficient of variation of answer lengths (measures length consistency). A golden set where all expected answers use the same 10 words with zero TTR variation is flagged for likely templating or automation — templates generate consistent-looking but informationally poor reference answers.

4.2 v0.4.0 — Near-Duplicate Scan

Added the pairwise near-duplicate scan with configurable Jaccard threshold. Added sampling mode for large golden sets (>5,000 examples) to prevent O(n²) timeout.

4.3 v0.3.0 — Category Coverage Check

Added category coverage validation. Added INSUFFICIENT_INPUT as a distinct return status for checks that cannot run without optional configuration (split_col, category_col).

4.4 v0.2.0 — Content Quality Checks

Added weak answer, ambiguous prompt, and over-easy detection. Added HTML output format.

4.5 v0.1.0 — Structural Checks

Initial release: conflicting labels (FAIL) and exact duplicates (WARN). JSON and Markdown output.

SECTION 5 — DESIGN DECISIONS

5.1 Why Audit the Dataset, Not the Model Outputs?

Most LLM evaluation tooling focuses on scoring model outputs against reference answers. GoldenSetAuditor is upstream of that: it checks whether the reference answers are trustworthy before any model is evaluated against them. The insight is that a bad reference answer invalidates every model score computed against it — not just the current model's score, but every future model's score as well. Fixing the reference is a permanent improvement; fixing a model score doesn't fix the underlying evaluation problem.

5.2 Why FAIL Only for Conflicting Labels?

Conflicting labels break the evaluation contract: there is no correct answer to a question that has two different correct answers in the dataset. Every other quality issue — weak answers, near-duplicates, ambiguous prompts — reduces the informativeness or efficiency of the evaluation without making it structurally invalid. FAIL is reserved for structural invalidity.

5.3 Why Include a Truth Boundary on Every Report?

The truth boundary text ("GoldenSetAuditor audits dataset quality; it does not evaluate model answers") cannot be removed from any output format. This prevents reports from being used as evidence that a golden set is suitable for evaluation — which it is not, since suitability requires domain expert review beyond what automated checks provide.

SECTION 6 — Q&A

Q: How is this different from just reviewing the golden set manually?

Manual review is inconsistent and undocumented. Two reviewers will catch different issues. The same reviewer will catch different issues on different days. GoldenSetAuditor produces a structured report with row-level evidence: "prompt at row 47 is an exact duplicate of row 12" or "category 'billing' has only 2 examples, below the minimum of 5." That evidence is attached to the model card, reviewable in a PR, and reproducible.

Q: What's the most common critical finding in practice?

Near-duplicates. Teams assembling golden sets manually often generate multiple phrasings of similar questions — "How do I reset my password?" and "What are the steps to reset my password?" look different but are nearly identical to a retrieval system. A golden set with 30% near-duplicate prompts is measuring how well the model handles one phrasing of 70% of the intended question space.

Q: Can GoldenSetAuditor catch contamination from pretraining data?

No. Contamination detection requires comparing the golden set against the model's training corpus, which GoldenSetAuditor doesn't have access to. Near-duplicate detection catches paraphrasing within the golden set itself, not against external corpora.

Q: How does it handle multi-turn evaluation datasets?

Multi-turn datasets (where each "prompt" is a conversation history) are partially supported: the conflicting label, exact duplicate, and weak answer checks work on the final turn's expected answer. Near-duplicate detection works on the full conversation hash. Ambiguous prompt detection may produce false positives (anaphoric pronouns are common in multi-turn conversations and are legitimate) — the `ambiguous_prompt` check can be disabled for multi-turn datasets via config.

Q: Where does it fit in the LLM development workflow?

GoldenSetAuditor runs as a pre-benchmark gate. The workflow is: (1) assemble golden set, (2) run GoldenSetAuditor, (3) resolve FAIL findings (conflicting labels must be fixed), (4) review and address WARN findings, (5) run model benchmark. Steps 2–4 happen before any model sees the evaluation data.

SECTION 7 — NUMBERS AT A GLANCE

Metric	Value
Checks	7
Checks that can FAIL	1 (conflicting labels only)
Checks that WARN	5
Checks with INSUFFICIENT_INPUT	2 (near-duplicate if n>5000 sampled; coverage if no category_col)
Current version	v0.5.0
Test cases	13
Output formats	JSON, Markdown, HTML

GoldenSetAuditor Interview Defense Document v2.0 — Complete Edition

Seven Checks · FAIL on Conflicting Labels · Answer Diversity · Pre-Benchmark Gate